

END-TO-END MLOPS PROJECT

Food Waste Level Predictor

Architecture and step-by-step guide: data validation, training with a quality gate, testing, FastAPI serving, Streamlit UI, Docker, GitHub Actions CI and drift monitoring.

Project folder	C:\Users\priya\MLOPS\food-waste-mlops
Dataset	AI-Powered Food Waste Management Dataset v1 (Kaggle, synthetic, 8,000 rows)
Best model	HistGradientBoosting - macro F1 0.865, accuracy 89.0%
Stack	Python 3.10, scikit-learn 1.7.2, FastAPI, Streamlit, Docker, GitHub Actions, pytest
Version	October 2026

Contents

	Section	What you will learn
	Architecture overview	How all the pieces fit together, and the three flows: build, serve, monitor
1	Environment setup	Virtual environment, dependencies, Jupyter kernel
2	Data and validation	The dataset, the target, data leakage, validation rules
3	Feature engineering and pipeline	Engineered features, one sklearn Pipeline for train and serve
4	Training and quality gate	Model comparison, macro F1, the gate that blocks bad models
5	Automated testing	21 pytest tests: data, model behaviour, API contract, drift
6	Serving with FastAPI	REST endpoints, input validation, prediction logging
7	Streamlit UI	The user-facing app and its fallback to the local model
8	Docker	Image that trains at build time, docker compose for API + UI
9	CI/CD with GitHub Actions	What runs on every push and why
10	Monitoring and data drift	KS and chi-square tests, the heatwave simulation
A-C	Appendices	Troubleshooting, command cheat sheet, next improvements

OVERVIEW

System architecture

What the system does, how its parts connect, and why it is built this way.

The problem

Restaurants, supermarkets, cafeterias and cloud kitchens order food every morning without knowing how much will be wasted by the evening. This system predicts, **at the start of the day**, whether today's stock for a business and food category will end with **Low, Medium or High waste**. That leaves time to act: order less, run a promotion, or plan a donation.

A model on its own is not enough to be useful in production. It has to be trained reproducibly, checked automatically, served reliably and watched after release. That is what MLOps adds, and each piece below covers one of those jobs.

Architecture diagram

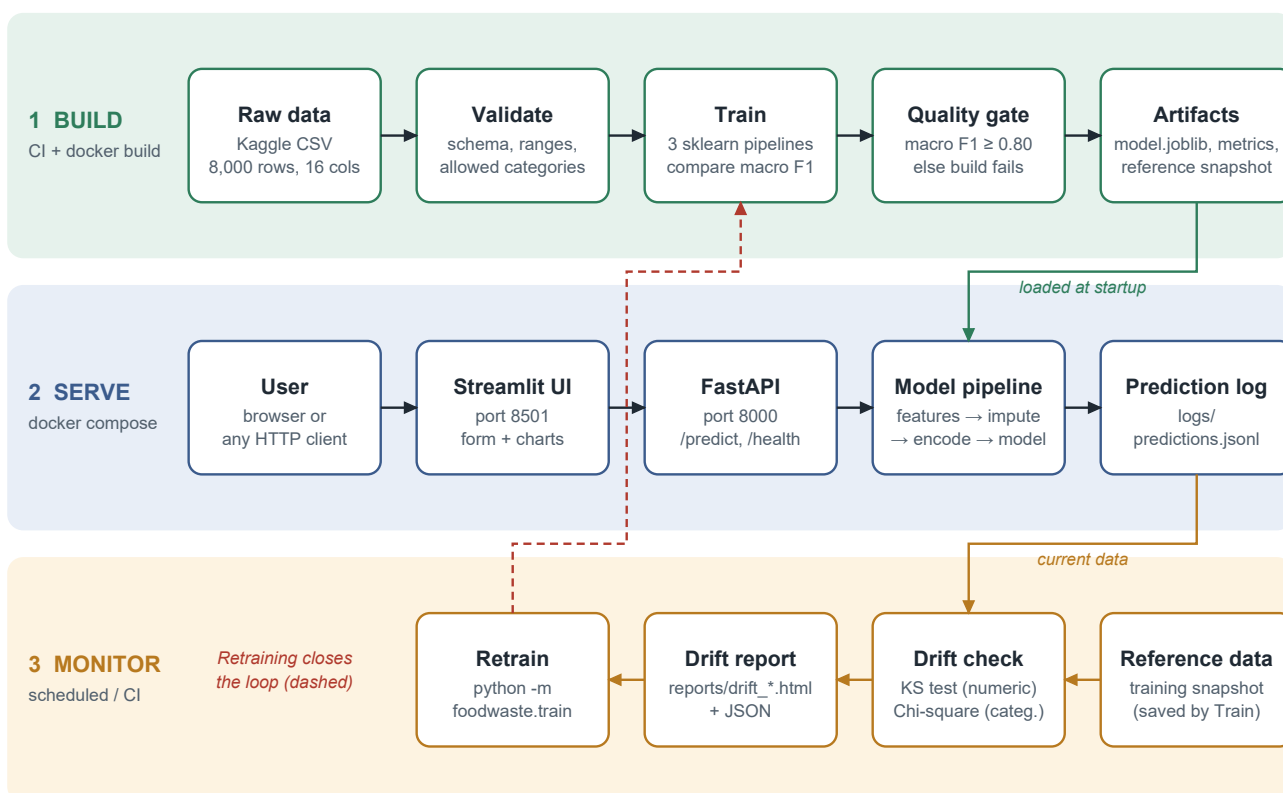


Figure 1 - Three lanes: Build creates a verified model, Serve answers requests, Monitor compares incoming data with the training snapshot and triggers retraining (dashed red) when it drifts.

The three flows

1. Build flow (runs in CI and during docker build)

Raw CSV $\rightarrow$ validation $\rightarrow$ three candidate pipelines trained and compared $\rightarrow$ the best one must reach macro F1 ≥ 0.80 $\rightarrow$ only then are the model, its metrics and a snapshot of the training data saved. If any step fails, nothing is saved and the build goes red.

2. Serve flow (runs in Docker containers)

A user fills in the Streamlit form → the UI sends JSON to FastAPI /predict → FastAPI validates the input, runs the saved pipeline, returns the level with probabilities and the model version → every request and prediction is appended to a log file.

3. Monitor flow (scheduled, or on demand)

The drift check compares new data (the API log, a CSV, or a simulated batch) against the training snapshot, feature by feature. It writes an HTML and JSON report. If drift is found, the model is retrained, which starts the build flow again.

Components and technology

Component	Technology	Responsibility	Code
Config	Python	Paths, feature lists, thresholds in one place	foodwaste/config.py
Validation	pandas	Reject bad data before it reaches the model	foodwaste/data.py
Training	scikit-learn	Pipelines, model comparison, quality gate, artifacts	foodwaste/train.py
Prediction	joblib	Load the model once and predict from raw records	foodwaste/predict.py
API	FastAPI, pydantic	REST endpoints, input schema, request logging	api/main.py
UI	Streamlit	Form, result, model metrics, drift demo	app/streamlit_app.py
Monitoring	SciPy	KS / chi-square drift tests, HTML reports	foodwaste/drift.py
Tests	pytest, httpx	21 automated checks	tests/
Container	Docker, compose	Reproducible runtime; trains at build time	Dockerfile, docker-compose.yml
CI/CD	GitHub Actions	Train, test, drift-check, build, smoke-test on every push	.github/workflows/ci.yml

Folder structure

```

food-waste-mlops/
├── data/                raw dataset + data dictionary (committed)
├── foodwaste/           core package - used by notebook, API, UI, CI and Docker
│   ├── config.py        single source of truth: paths, features, thresholds
│   ├── data.py          load, validate, feature engineering
│   ├── train.py         train + compare + quality gate + save artifacts
│   ├── predict.py       load model, predict from raw records
│   └── drift.py         drift detection + HTML/JSON reports
├── api/main.py          FastAPI service
├── app/streamlit_app.py Streamlit UI
├── tests/               pytest suite (data, model, api, drift)
├── notebooks/walkthrough.ipynb
├── models/ logs/ reports/ generated - not committed to git
├── Dockerfile docker-compose.yml .dockerignore
├── .github/workflows/ci.yml
├── requirements.txt requirements-dev.txt pytest.ini README.md

```

Design principles

- **One codebase, many entry points.** The notebook, API, UI, tests, CI and Docker all import the same foodwaste package, so there is never a 'notebook version' and a 'production version' that behave differently.
- **All preprocessing lives inside the model pipeline.** Feature engineering, imputation and encoding are saved together with the model. The API sends raw fields, so training and serving cannot drift apart.
- **No data leakage.** Only information available in the morning is used as input.
- **Fail loudly and early.** Bad data, a weak model, a broken API or a failing container all stop the pipeline instead of reaching users.
- **Artifacts are rebuilt, not stored.** Models, logs and reports are excluded from git; any machine can recreate them from code + data.
- **Every prediction is traceable.** Responses carry a model version; metrics.json records the data hash, library version and scores of the run that produced it.

STEP 1

Environment setup

Create an isolated Python environment with exactly the libraries the project needs.

What and why

Each project gets its own **virtual environment** (venv) so its library versions do not clash with other projects. scikit-learn is **pinned to 1.7.2** because a saved model must be loaded with the same version that trained it; a mismatch produces warnings or wrong results.

Files

File	Contents
requirements.txt	Runtime libraries: numpy, pandas, scikit-learn==1.7.2, scipy, joblib, matplotlib, fastapi, uvicorn, pydantic, streamlit, requests
requirements-dev.txt	Everything above plus pytest, httpx (API tests), jupyter, ipykernel

How to run

```
cd C:\Users\priya\MLOPS\food-waste-mlops
python -m venv venv
.\venv\Scripts\Activate.ps1          # prompt now starts with (venv)
pip install -r requirements-dev.txt

# make the venv available as a Jupyter kernel
python -m ipykernel install --user --name food-waste-venv --display-name "Python (food-waste venv)"
```

Expected result

The terminal prompt shows (venv), and in Jupyter the kernel **Python (food-waste venv)** appears in the kernel list.

Watch out

In Jupyter, always check the kernel name in the top-right corner. The default **Python 3** kernel uses the system Python (C:\Python310), which has different library versions. That caused the 'Trying to unpickle estimator from version 1.7.1' warning in the earlier IMDB project.

Why it matters

Reproducibility starts here. Pinned versions plus a clean environment mean the model trained on your laptop, in CI and inside Docker is built from the same code with the same libraries.

STEP 2

Data understanding and validation

Know what each column means, decide which ones the model may use, and stop bad data at the door.

The dataset

8,000 daily inventory records (Jan-Dec 2025) for four business types and six food categories. It is synthetic but generated from a causal model: demand → purchasing → sales → leftover stock → waste. About 3% of values are missing in five columns, and about 1% of order quantities are outliers.

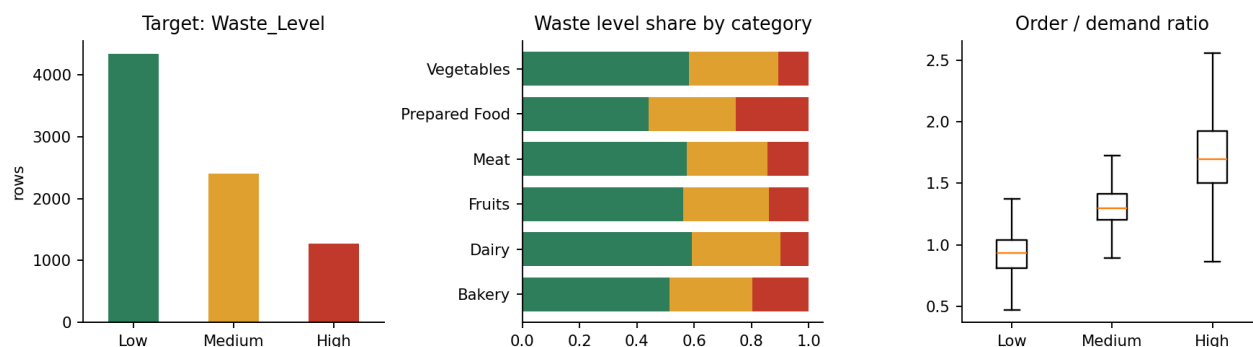


Figure 2 - Left: the target is imbalanced (54% Low, 30% Medium, 16% High). Middle: share of each waste level by category. Right: ordering more than demand is strongly linked to higher waste.

Target and inputs

Role	Columns	Reason
Target	Waste_Level (Low / Medium / High)	What we predict
Inputs (known in the morning)	Date, Business_Type, Food_Category, Purchase_Quantity, Shelf_Life_Days, Storage_Temperature, Daily_Demand, Promotion, Weather	Available when the order is placed
Excluded: leakage	Units_Sold, Remaining_Stock, Waste_Quantity, Waste_Reason, Donation_Made	Only known after the day ends, or computed from the waste itself
Excluded: ID	Inventory_ID	Unique per row, carries no signal

Data leakage

If a column that is only known *after* the outcome is used as an input, the model looks excellent in testing and fails in real use. The notebook demonstrates this: a simple tree using `Waste_Quantity` / `Purchase_Quantity` reaches **96.7% accuracy**, but nobody knows the waste quantity in the morning. The leakage columns are listed in `config.py` and a test fails if any of them ever appears in the feature list.

Assumption

`Daily_Demand` is borderline: the dataset defines it as the demand that actually happened. This project treats it as the business's demand forecast, which is a reasonable assumption, and the README says so openly. Stating assumptions like this is part of good ML practice.

Validation rules

`validate()` in `foodwaste/data.py` runs before training *and* before every prediction. It collects every problem and raises one clear error:

Rule	Example of rejected data
All required columns exist	File without Food_Category
Categories are from the allowed list	Food_Category = 'Seafood'
Purchase_Quantity > 0	Purchase_Quantity = -10
Shelf_Life_Days ≥ 1, Daily_Demand ≥ 0	Shelf_Life_Days = 0
Date parses as a date	Date = 'not-a-date'
Target has no missing values (training only)	Empty Waste_Level

```
>>> validate(bad)
DataValidationError: Food_Category: unknown values ['Seafood']; Purchase_Quantity must be > 0
```

Why it matters

Most production ML failures are data failures, not model failures. A validation layer turns silent garbage-in/garbage-out into a loud, explainable error.

STEP 3

Feature engineering and preprocessing pipeline

Turn raw columns into model-ready numbers, inside one object that is saved with the model.

Engineered features

Feature	Formula	Why
Month	month of Date	Seasonality
Day_Of_Week	0 = Monday ... 6 = Sunday	Weekly demand pattern
Is_Weekend	Day_Of_Week $\geq$ 5	Weekend effect
Order_To_Demand_Ratio	Purchase_Quantity / Daily_Demand	Over-ordering is the main driver of waste (see Figure 2, right)

The pipeline

Everything is chained in one scikit-learn Pipeline:

```
Pipeline([
    ("features", FunctionTransformer(add_features)),          # Month, Day_Of_Week, ratio ...
    ("preprocess", ColumnTransformer([
        ("num", SimpleImputer(median) -> StandardScaler,      numeric columns),
        ("cat", SimpleImputer(most_frequent)
            -> OneHotEncoder(handle_unknown="ignore"),          categorical columns),
    ])),
    ("model", <classifier>),
])
```

How missing values and outliers are handled

- **Missing numbers** (Shelf_Life_Days, Storage_Temperature, Daily_Demand) are filled with the training median. Missing Weather is filled with the most frequent value.
- **Unknown categories** at prediction time are ignored by the one-hot encoder instead of crashing (the API already rejects them, so this is a second safety net).
- **Outliers** in Purchase_Quantity are kept. They are real bulk orders, and tree models are robust to them.
- **Class imbalance** is handled with `class_weight="balanced"` so the rarer High class is not ignored.

Why it matters

Because preprocessing is part of the saved model, the API only needs raw fields. There is no second copy of the feature code that could get out of sync with training. This mismatch, called **training-serving skew**, is one of the most common production ML bugs.

STEP 4

Model training, comparison and quality gate

Train several candidates, pick the best on the right metric, and refuse to ship a weak model.

What happens when you run training

1. Load and validate the CSV.
2. Stratified 80/20 split (6,400 train / 1,600 test). Stratified means each split keeps the 54/30/16 class mix.
3. Train three pipelines: Logistic Regression, Random Forest, HistGradientBoosting.
4. Score each on the test set with **macro F1** and accuracy; pick the best by macro F1.
5. **Quality gate:** if the best macro F1 is below 0.80, stop with exit code 1 and save nothing.
6. Otherwise save the model, metrics.json, the confusion matrix and a snapshot of the training data.

```
python -m foodwaste.train
```

```
logistic_regression    macro F1 0.8236    accuracy 0.8562
random_forest          macro F1 0.8451    accuracy 0.8738
hist_gradient_boosting macro F1 0.8648    accuracy 0.8900
```

Best model: hist_gradient_boosting (macro F1 0.8648)

Quality gate passed (≥ 0.8). Saved model version 20261004-093839 to models/model.joblib

Results

Class	Precision	Recall	F1	Test rows
Low	0.94	0.94	0.94	866
Medium	0.82	0.86	0.84	480
High	0.85	0.78	0.81	254
Macro average	0.87	0.86	0.86	1,600

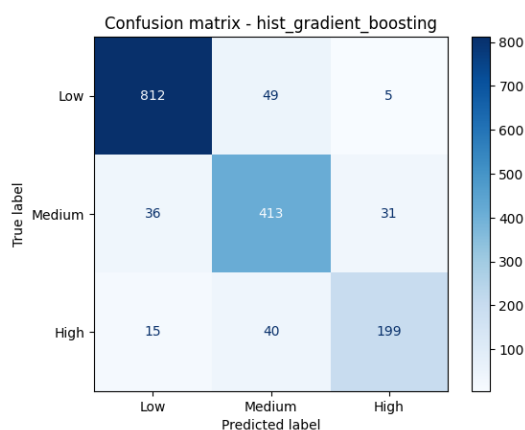


Figure 3 - Confusion matrix of the best model on the 1,600 test rows. Most errors are between neighbouring levels (High vs Medium).

Why macro F1 and not accuracy?

A model that always answers 'Low' already scores 54% accuracy while being useless. **Macro F1** averages the F1 score of each class equally, so it only goes up if the model also gets Medium and High right. A test also checks that accuracy beats the always-Low baseline.

Artifacts written to models/

File	Contents	Used by
model.joblib	The full fitted pipeline	API, UI, notebook
metrics.json	Version, scores of every candidate, per-class report, data MD5 hash, sklearn version, feature list	API /model-info, UI Model tab
confusion_matrix.png	Chart above	UI Model tab
reference_data.csv	The training rows	Drift monitoring baseline

Why it matters

The quality gate turns 'we hope the new model is good' into an automatic rule. In CI and in docker build, a model below the threshold makes the whole build fail, so it can never be deployed.

STEP 5

Automated testing

21 pytest tests that check the data, the model's behaviour, the API contract and the drift logic.

Test suite

File	What it checks
test_data.py	Real data passes validation · no leakage column is used as a feature · missing columns, unknown categories and non-positive quantities are rejected · engineered features are created correctly
test_model.py	Model passes the quality gate · beats the always-Low baseline · only predicts Low/Medium/High · handles missing values · behaviour test : ordering far more than demand never lowers the High-waste probability
test_api.py	/health returns ok · /predict returns a valid level and probabilities summing to 1 · each request is logged · batch endpoint works · optional fields can be omitted · invalid category, negative quantity and bad date return HTTP 422
test_drift.py	A normal batch shows no drift (no false alarms) · a heatwave batch is detected on Weather and Storage_Temperature

How to run

In a terminal (venv active), from the project folder:

```
pytest          # short output
pytest -v       # one line per test
```

Inside a Jupyter notebook cell, prefix it with ! and use the kernel's Python:

```
import os, sys
os.chdir(r"C:\Users\priya\MLOPS\food-waste-mlops")
!{sys.executable} -m pytest -v
```

Expected result

```
..... [100%]
21 passed in 10s
```

Watch out

Typing pytest alone in a notebook cell gives `NameError: name 'pytest' is not defined`. A cell runs Python code, and pytest is a terminal command. Use the ! form above.

Why it matters

ML tests go beyond 'does the code run'. Behaviour tests (more over-ordering should mean more waste risk) catch models that score well but have learned the wrong thing. The API tests act as a contract with whoever calls the service.

STEP 6

Model serving with FastAPI

Expose the model as a REST API with a strict input schema, versioned responses and request logging.

Endpoints

Method	Path	Purpose
GET	/health	Returns {"status": "ok"} when the model is loaded (503 otherwise). Used by Docker's HEALTHCHECK and the CI smoke test.
GET	/model-info	Model version, best model, scores, data hash, sklearn version
POST	/predict	One record in, one prediction out
POST	/predict/batch	A list of records in, a list of predictions out
GET	/docs	Interactive Swagger UI generated automatically

Run it

```
uvicorn api.main:app --reload          # then open http://localhost:8000/docs
```

Example request and response

```
POST /predict
{
  "Date": "2025-07-14", "Business_Type": "Restaurant", "Food_Category": "Prepared Food",
  "Purchase_Quantity": 180, "Shelf_Life_Days": 1, "Storage_Temperature": 24,
  "Daily_Demand": 110, "Promotion": "No", "Weather": "Rainy"
}

200 OK
{"waste_level": "High",
 "probabilities": {"Low": 0.0005, "Medium": 0.0031, "High": 0.9964},
 "model_version": "20261004-093839"}
```

Ordering 180 units against an expected demand of 110 gives **High** (99.6%). The same request with Purchase_Quantity 110 gives **Low** (95.6%).

Input validation

The request body is a pydantic model. Categories must come from the allowed lists, Purchase_Quantity must be > 0, Shelf_Life_Days ≥ 1, and Date must be a real date. Anything else is rejected with **HTTP 422** and a message naming the bad field, before the model runs. Shelf life, temperature, demand and weather are optional because the pipeline can fill them in.

Prediction logging

Every request is appended to logs/predictions.jsonl (timestamp, input, prediction). This log is the 'current data' that the monitoring step compares with the training data.

Why it matters

A service answer always says which model version produced it. When a prediction is questioned later, you can trace it back to the exact training run, data hash and scores in metrics.json.

STEP 7

Streamlit user interface

A simple app for non-technical users, built on top of the API.

Tabs

Tab	What it shows
Predict	Form with date, business type, food category, quantity, demand, shelf life, temperature, weather and promotion → predicted level with colour, advice, probability chart, and 'Served by' + model version
Model	Best model, version, rows, quality gate, scores of all three candidates, confusion matrix
Monitoring	Run a drift check on a normal or heatwave batch and see which features drifted

How it talks to the model

The app sends the form to `API_URL/predict` (default `http://localhost:8000`, or `http://api:8000` inside docker compose). If the API cannot be reached, it falls back to loading the model file directly, and the caption changes to *'local model (API offline)'*, so the app still works for a demo.

Run it

```
streamlit run app/streamlit_app.py # http://localhost:8501
```

Advice shown for each level

Level	Advice
Low	Order looks right for expected demand.
Medium	Some waste likely - consider a smaller order or a promotion.
High	High waste risk - cut the order, run a promotion or plan a donation.

Watch out

Port 8501 can only be used by one app at a time. If an older Streamlit app (for example the IMDB project) is still running, stop it with Ctrl+C in its terminal, or start the other app with `--server.port 8502`.

STEP 8

Containerisation with Docker

Package the code, libraries, data and a freshly trained model into one image that runs anywhere.

The Dockerfile

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt # cached until requirements change
COPY foodwaste/ api/ app/ data/food_waste_dataset.csv ...
RUN python -m foodwaste.train # trains in the image; fails below gate
EXPOSE 8000
HEALTHCHECK CMD python -c "urllib.request.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Key decisions

- **Train at build time.** The image always contains a model that passed the quality gate, built from the exact code and data in that image.
- **Layer order.** Requirements are installed before the code is copied, so editing code does not reinstall every library. That is why a rebuild takes about 11 seconds.
- **.dockerignore** keeps venv/, tests, notebooks, logs and old models out of the image.
- **HEALTHCHECK** lets Docker report the container as healthy only once the API answers.

docker compose: API + UI

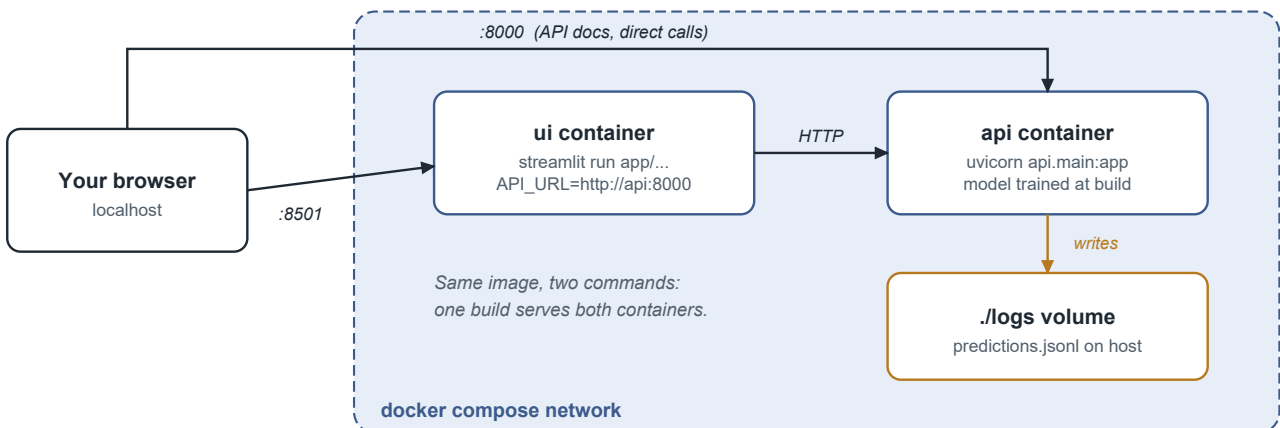


Figure 4 - One image, two containers. The UI reaches the API by its service name (http://api:8000). Logs are written to a folder on your machine.

```
docker compose up --build # first time, or after code changes
docker compose up        # later starts
docker compose down      # stop and remove containers
```

Then open the UI at <http://localhost:8501> and the API docs at <http://localhost:8000/docs>. In the UI, 'Served by: FastAPI service' confirms the two containers are talking.

Troubleshooting: port already in use

Error: 'ports are not available: listen tcp 0.0.0.0:8501: bind: Only one usage of each socket address...'

Cause: something on your machine already uses that port (here, the old IMDB Streamlit app on 8501 and a local uvicorn on 8000).

Fix: stop those processes (Ctrl+C in their terminals), then run `docker compose up` again.

STEP 9

CI/CD with GitHub Actions

Every push is automatically trained, tested, drift-checked, containerised and smoke-tested.

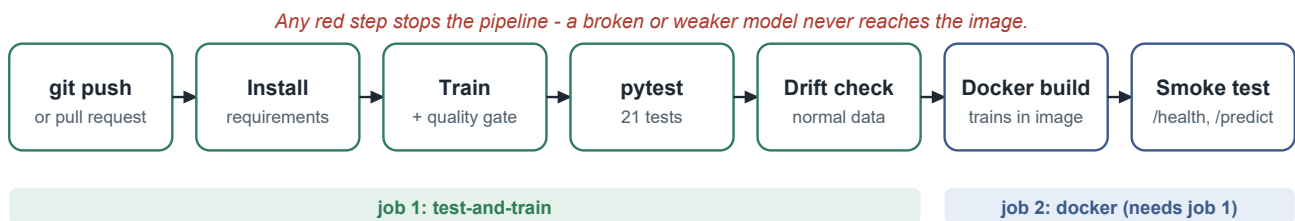


Figure 5 - The workflow in `.github/workflows/ci.yml`. Job 2 only starts if job 1 passes.

Steps

Step	Command	Fails when
Install	<code>pip install -r requirements.txt pytest httpx</code>	A dependency cannot be installed
Train	<code>python -m foodwaste.train</code>	Data is invalid or macro F1 < 0.80
Test	<code>pytest -v</code>	Any of the 21 tests fails
Drift check	<code>python -m foodwaste.drift --simulate normal --fail-on-drift</code>	Normal data is flagged (a false alarm means the check is broken)
Upload	<code>actions/upload-artifact</code>	- (saves metrics, confusion matrix, reports)
Docker build	<code>docker build -t food-waste-api .</code>	Image cannot be built or training inside fails
Smoke test	<code>docker run + curl /health + curl /predict</code>	Container does not start or answer

Put it on GitHub

```

cd C:\Users\priya\MLOPS\food-waste-mlops
git init
git add .
git commit -m "Food waste MLOps project"
git branch -M main
git remote add origin https://github.com/<your-username>/food-waste-mlops.git
git push -u origin main
  
```

Open the repository on GitHub and go to the **Actions** tab to watch the run. A green tick means every gate passed.

Why it matters

This is the 'Ops' in MLOps: no manual step decides whether a model is good enough. The same automatic checks run for every change, by anyone, on a clean machine.

STEP 10

Monitoring and data drift

Detect when the data the model sees in production no longer looks like the data it was trained on.

What is data drift?

A model learns patterns from its training data. If the real world changes, for example a heatwave, a new supplier, or a change in how staff record data, the inputs shift and predictions silently get worse. Drift monitoring compares new inputs with the training snapshot so the change is noticed *before* it causes damage.

How the check works

Feature type	Test	Effect size	Flagged as drift when
Numeric (4)	Kolmogorov-Smirnov	KS statistic	$p < 0.05$ and $KS \geq 0.10$
Categorical (4)	Chi-square	Total variation distance (TVD)	$p < 0.05$ and $TVD \geq 0.10$

Both conditions are required. With thousands of rows a statistical test flags even tiny, harmless differences, and the effect-size threshold makes sure the change is big enough to matter.

Run it

```
python -m foodwaste.drift --simulate normal      # fresh sample, same conditions -> no drift
python -m foodwaste.drift --simulate heatwave    # 70% hot days, storage ~6 C warmer -> drift
python -m foodwaste.drift --from-logs           # requests received by the API
python -m foodwaste.drift --current new.csv      # any CSV with the input columns
# add --fail-on-drift to exit with code 1 (for schedulers and CI)
```

Result of the heatwave simulation

Feature	Statistic	p-value	Drifted
Storage_Temperature	KS 0.385 (mean 10.6 → 16.5 °C)	2.3e-113	Yes
Weather	TVD 0.538 (Hot 20% → 74%)	1.2e-271	Yes
The other 6 features	≤ 0.034	> 0.27	No

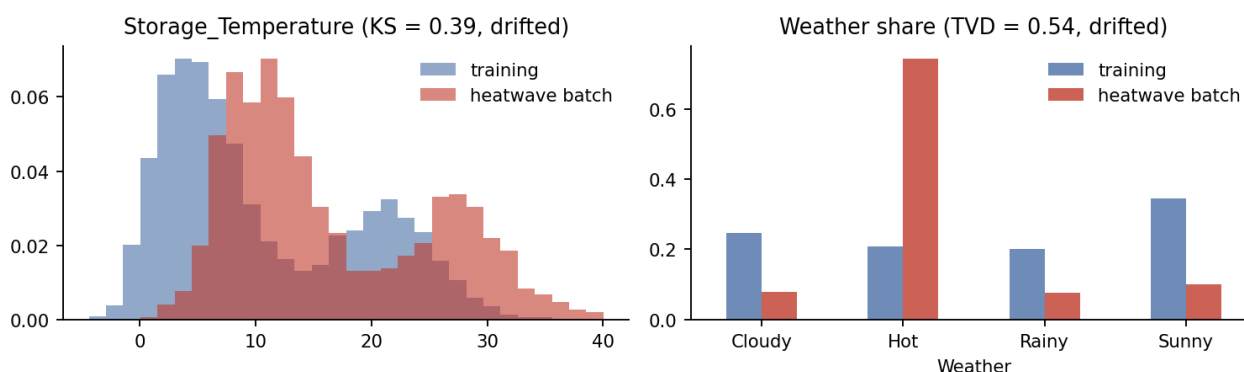


Figure 6 - The heatwave batch (red) against the training data (blue). These are the charts embedded in `reports/drift_heatwave.html`.

Reports

Each run writes `reports/drift_<name>.html` (status, table, charts of the drifted features) and a matching JSON file for machines. Open the HTML file in a browser.

Watch out

Drift tests need enough data. With fewer than 100 rows the script prints a warning: a handful of identical test requests will always look 'different' from 6,400 training rows.

Why it matters

Monitoring closes the MLOps loop: build → serve → monitor → retrain. In a real deployment the `--from-logs` check would run on a schedule, and `--fail-on-drift` would trigger retraining or an alert.

APPENDIX A

Troubleshooting

Problems met while building this project, and how they were fixed.

Symptom	Cause	Fix
NameError: name 'pytest' is not defined (in a notebook)	pytest is a terminal command, not Python	Use <code>!{sys.executable} -m pytest</code> , or run it in a terminal
InconsistentVersionWarning: unpickle estimator from version 1.7.1 when using 1.7.2	Notebook used the system Python kernel, not the venv	Select kernel 'Python (food-waste venv)' and retrain
Port 8501 shows the IMDB app	Old Streamlit process still running	Ctrl+C in its terminal, or run the other app on <code>--server.port 8502</code>
docker compose: ports are not available ... bind	Port 8000 or 8501 already in use on the host	Stop local uvicorn / streamlit, then docker compose up
Docker: cannot find dockerDesktopLinuxEngine	Docker Desktop is not started	Start Docker Desktop and wait until it is running
/health returns 503	No model file yet	<code>python -m foodwaste.train</code>
UI says 'local model (API offline)'	API not running or wrong API_URL	Start uvicorn or docker compose
Opened the wrong project folder in Jupyter	Several similar folders under MLOPS	Use MLOPS/food-waste-mlops/notebooks/walkthrough.ipynb
A server cell freezes the notebook	uvicorn / streamlit run until stopped	Start servers in a terminal, not a notebook cell

APPENDIX B

Command cheat sheet

Everything you need, in the order you normally use it.

```
# 0. once per terminal
cd C:\Users\priya\MLOPS\food-waste-mlops
.\env\Scripts\Activate.ps1

# 1. train (with quality gate)
python -m foodwaste.train

# 2. test
pytest -v

# 3. serve
uvicorn api.main:app --reload          # API  http://localhost:8000/docs  (terminal 1)
streamlit run app/streamlit_app.py     # UI   http://localhost:8501      (terminal 2)

# 4. monitor
python -m foodwaste.drift --simulate heatwave
python -m foodwaste.drift --from-logs

# 5. containers
docker compose up --build
docker compose down

# 6. ship
git add . ; git commit -m "message" ; git push  # GitHub Actions runs the CI pipeline
```

APPENDIX C

Next improvements

- **Experiment tracking and model registry** with MLflow: log every run's parameters, metrics and model, and promote versions from Staging to Production.
- **Automated retraining**: a scheduled GitHub Action that runs the drift check on the prediction log and retrains when drift is found.
- **Cloud deployment**: push the image to a registry (Docker Hub, AWS ECR) and run it on a managed service (AWS App Runner / ECS, Azure Container Apps, Google Cloud Run).
- **Performance monitoring**: once the real waste level is known each evening, join it with the logged predictions and track live macro F1, not only input drift.
- **Data versioning** with DVC, so every model can be traced to an exact dataset version.
- **Hyperparameter tuning** (GridSearchCV or Optuna) inside the training step, still behind the quality gate.